



EuroHPC Joint Undertaking (JU) Development Access

Final Report

1 General Information

Type of project granted: Development Access

1.1 Proposal ID

Please fill in the information in the box below (e.g. EHPC-DEV-202XDXX-XXX).

EHPC-DEV-2026D03-189

1.2 Industry and Public Sector Involvement, if applicable

Please click once in the box to select it, click again to unselect.

- ☐ Industry involvement
- ☐ Public Sector involvement

Please explain the level of involvement (As PI, Team member or both) below.

Purely academic research without industry or public sector involvement.

1.3 Period of access to the EuroHPC JU facilities

Please fill in the information in the box below.

Start date of the allocation (DD/MM/YYYY)	18/03/2026
---	------------



End date of the allocation (DD/MM/YYYY)	18/03/2027
Duration of extension in months (if applicable)	0

1.4 EuroHPC JU systemassigned

Please click once in the box to select it, click again to unselect.

- | | |
|---|---|
| ☐ Vega CPU (IZUM) | ☐ Vega GPU (IZUM) |
| ☐ MeluXina CPU (LuxProvide) | ☐ MeluXina GPU (LuxProvide) |
| ☐ MeluXina FPGA (LuxProvide) | ☐ Discoverer GPU (SofiaTech) |
| ☐ Discoverer CPU (SofiaTech) | ☐ Deucalion ARM (FCT) |
| ☐ Deucalion x86 (FCT) | ☐ Deucalion GPU (FCT) |
| ☐ LUMI-C (CSC) | ☐ LUMI-G (CSC) |
| ☒ Leonardo Booster (CINECA) | ☐ Leonardo DCGP (CINECA) |
| ☐ MN5 GPP (BSC) | ☐ MN5 ACC (BSC) |
| ☐ MN5 HBM (BSC) | ☐ MN5 NG-GPP (BSC) |
| ☐ Jupiter Booster (JSC) | ☐ Arrhenius GPU (NAISS) |
| ☐ Arrhenius CPU (NAISS) | |

1.5 Principal Investigator



Please fill in the information in the table below.

Title	Dr.
First (Given) Name	Kosmas
Last (Family) Name	Kritsis
E-mail Address	kosmas.kritsis@athenarc.gr

2 Project information

2.1 Project title

Please fill in the information in the box below.

MAS4CS: A Multi-Agent System for Customer Service Automation using Structured LLM Architectures and Role-Specific Fine-Tuning (implemented and evaluated as MLP4CS: Modular LLM Pipeline for Task-Oriented Customer Service Dialogue).

2.2 Main research field(s)

Please click once in the box to select it, click again to unselect.

- | | |
|---|---|
| ☐ Biochemistry, Bioinformatics and Life sciences | ☐ Fundamental Constituents of Matter |
| ☐ Chemical Sciences and Materials | ☐ Linguistics, Cognition and Culture |
| ☐ Earth System Sciences | ☒ Mathematics and Computer Sciences |
| ☐ Economics, Finance and Management | ☐ Physiology and Medicine |
| ☐ Engineering | ☐ Universe Science |
-



2.3 Artificial Intelligence technologies used, if applicable

Please click once in the box to select it, click again to unselect.

- | | |
|---|---|
| ☒ Machine Learning | ☐ Audio (speech recognition, speech synthesis, etc.) |
| ☒ Natural Language Processing | ☐ Robotic process automation |
| ☒ Generative Language Modeling | ☒ Virtual agents |
| ☒ Deep Learning | ☐ Decision management: Classified and statistical learning methods |
| ☐ Vision (image recognition, image generation, text recognition OCR, etc.) | ☐ Other* |

* Other - Specify the used AI technology:

2.4 Team members and institutions

Please list all team members and corresponding affiliations that were involved in the project.

- | |
|---|
| <ol style="list-style-type: none">1. Dr. Kosmas Kritsis (Principal Investigator), Institute for Language and Speech Processing (ILSP), Athena Research Center2. Georgios Xydias (Applicant / Researcher), National and Kapodistrian University of Athens |
|---|

2.5 Summary of the project

Please fill in the field with the same text used in the application form (maximum 300 words).



This project evaluates MLP4CS (Modular LLM Pipeline for Customer Service), a task-oriented customer service system for hospitality domains evaluated on MultiWOZ 2.2. The system structures dialogue into modular components of Dialogue State Tracking (DST) and Response Generation (ResponseGen) that are connected by deterministic database lookups, policy enforcement, and guardrails.

Three controlled experiments isolate the effects of architectural decomposition and role specialization: (1) a monolithic single-LLM baseline, (2) a zero-shot modular pipeline with homogeneous and heterogeneous setups, and (3) role-specific fine-tuning using QLoRA. Evaluated open-source models span leading families (Llama, Qwen, Phi) from 3B to 14B parameters. The objective is to determine whether decomposing sub-tasks improves performance across both standardized MultiWOZ benchmarks and custom operational metrics (covering hallucination, policy adherence, latency, and cost) over a single LLM, and whether parameter-efficient fine-tuning allows open-source models to match commercial APIs.

The EuroHPC resources were used to run controlled inference across all architectures and to train role-specific LoRA adapters for Experiment 3 on Leonardo Booster's NVIDIA A100 GPUs.

3 Main features of the code

3.1 Name of the code(s)

Please fill in the information in the box below.

MLP4CS (MAS4CS), PyTorch, Hugging Face Transformers, Unsloth.

3.2 Type of the code distribution

Please fill in the information in the box below (e.g., open source, commercial, academic, etc.).

Open source (MIT License)

3.3 Computational problem executed

Please fill in the information in the box below (e.g., N-body problem, Navier-stokes equation, etc.).



Parameter-efficient fine-tuning (QLoRA) of open source LLMs (3B to 14B parameters) and multi-turn inference for dialogue state tracking and response generation.

3.4 Computational method

Please fill in the information in the box below (e.g., FEM, FVM, PIC, spectral methods, etc.).

Task-Oriented Dialogue (TOD) in Natural Language Processing: Parameter-Efficient Fine-Tuning (PEFT) using QLoRA (4-bit NormalFloat base quantization with 16-bit LoRA adapters targeting all attention and MLP projections) trained via supervised instruction tuning (SFT) with AdamW 8-bit optimizer and cross-entropy loss. Autoregressive greedy decoding (temperature=0, do_sample=False) on NVIDIA A100 GPUs for multi-turn Dialogue State Tracking (DST) and delexicalized response generation.

3.5 Kind of parallelism used

Please fill in the information in the box below (e.g., MPI, OpenMP, MPI/OpenMP, pthreads, embarrassingly parallelism, etc.).

Single-GPU execution per job on NVIDIA A100 (64 GB) nodes with multi-processing/multi-threading for PyTorch DataLoader workers, batch gradient accumulation (effective batch size 8 via per-device batch size 2 x gradient accumulation 4), and embarrassingly parallel SLURM batch job arrays across model configurations and evaluation splits.

3.6 Main libraries used, version and language. Usage of /usr/local libraries.

Please fill in the information in the box below: main libraries (e.g., FFTW, MKL, BLAS, LAPACK, etc.), language (e.g., Fortran, C, C++, etc.).



Language: Python 3.10
Environment: Miniconda virtual environment

Main libraries:

- PyTorch 2.7.0 (CUDA backend)
- Hugging Face transformers 4.52.4
- unsloth 2025.11.1
- peft 0.19.0
- trl 0.23.0
- bitsandbytes 0.49.2
- accelerate 1.13.0
- datasets 4.8.4
- tokenizers 0.21.4
- mwzeval (commit cd3f0ee)

Usage of /usr/local libraries: None. All runtime libraries and dependencies were managed inside an isolated Miniconda environment, relying only on Leonardo CINECA's system NVIDIA GPU display drivers and kernel modules.

3.7 Other software used on the EuroHPC JU systems. Usage of post-processing or pre-processing tools.

Please fill in the information in the box below.

- SLURM Workload Manager: Used for batch scheduling and allocating single-GPU compute jobs via Bash submission scripts (.sh).
- Pre-processing scripts: Custom Python scripts to filter the MultiWOZ 2.2 dataset and construct role-specific training and inference datasets for Dialogue State Tracking and Response Generation.
- Post-processing scripts: Python evaluation scripts (including mwzeval) to parse model outputs and calculate task-oriented benchmark and operational metrics.

4 Compilation step

4.1 How is the program compiled?

Please fill in the information in the box below (e.g., makefile, script, etc.).

The codebase is written entirely in Python (Python 3.10) and did not require ahead-of-time compilation (e.g., makefile). All required packages and pre-compiled GPU-accelerated libraries (PyTorch with CUDA support, Unsloth, Transformers, and BitsAndBytes) were installed directly into an isolated Miniconda environment via shell scripts.



4.2 Difficulties met to compile, if any, and how they were tackled

Please fill in the information in the box below.

As noted in the CINECA technical assessment, a modern Python AI stack was not pre-installed system on Leonardo Booster. We resolved this by installing Miniconda in user space, setting up an isolated virtual environment, and pinning mutually compatible versions of PyTorch (2.7.0), Unsloth (2025.11.1), and BitsAndBytes (0.49.2) to match cluster GPU drivers without system-level conflicts.

4.3 Which version of the compiler and version of the MPI library was used?

Please fill in the information in the box below.

Compiler: N/A (the project used interpreted Python 3.10; system GCC 11.x and host NVIDIA CUDA drivers supported underlying pre-built PyTorch runtime binaries).
MPI Library: N/A (Standard MPI was not used; workloads ran on single-GPU instances per SLURM job).

4.4 Were any tools for studying the behavior of the code used?

Please fill in the information in the box below (e.g., debugger, profiler, etc.).

Yes. Unsloth's built-in memory diagnostics were used to monitor GPU VRAM allocation and reserved memory overhead during model loading, training, and inference. Hugging Face TRL / SFT Trainer logs were tracked to monitor training and validation loss curves with early stopping checkpoints. After job completion, CINECA SLURM commands were used to check GPU execution time, job efficiency, and remaining account quota.

5 Execution step

5.1 How is the program launched?

Please fill in the information in the box below.

Jobs were submitted to the Leonardo Booster compute partition via the SLURM workload manager using shell submission scripts (`sbatch job_script.sh`). Each SLURM script activated the virtual environment and ran the appropriate Python (.py) scripts for training or inference. Jobs requested single-GPU resources on NVIDIA A100 nodes.



5.2 Difficulties met to launch the code, if any, and how they were tackled

Please fill in the information in the box below.

No major difficulties were encountered when launching batch jobs on Leonardo Booster. The SLURM workload manager dispatched the single-GPU job scripts reliably with minimal queue wait times.

6 Communication patterns

If you know which are the main communication patterns used in your code configuration, please select the ones from the mentioned below (click once in the box to select it, click again to unselect):

- ☒ Many / few / no point-to-point communications
- ☒ Many / few / no collective communications
- ☐ Barrier
- ☐ Reduction
- ☐ Broadcast
- ☐ Scatter/gather
- ☐ All to all

7 Scalability testing

7.1 Summary of the obtained results from the scalability testing

Please show the scaling behavior of the application. Which progress was achieved? Does it fulfill the set expectations? If not, what were the reasons? (maximum 500 words)



In this project, scalability was evaluated along two dimensions: model parameter scale (ranging from 3B up to 14B parameters) and prompt context length under architectural decomposition (monolithic full-context in Experiment 1 versus modular partitioned prompts in Experiments 2 and 3).

Across parameter scales on Leonardo Booster (NVIDIA A100 64GB), inference throughput and memory scaling adhered closely to theoretical expectations under 4-bit NormalFloat (NF4) quantization. In the monolithic baseline (Experiment 1), prompt lengths frequently exceeded 20,000 tokens due to the serialization of full domain databases alongside multi-turn conversation history. This produced substantial per-turn latency: 7.82s for Qwen3-4B, 7.72s for Qwen3-8B, and climbing to 12.40s for Qwen3-14B and 13.81s for Phi-4-14B (the latter requiring 2x RoPE context extension beyond its 16K native limit).

Transitioning to the modular pipeline (Experiment 2) demonstrated strong computational efficiency gains. By isolating dialogue state tracking (DST) and response generation (ResponseGen) into focused per-role prompts (capped at 2,048 tokens), per-turn latency dropped by a factor of 3x to 5x across all open-source models (homogeneous configurations): Qwen3-4B fell from 7.82s to 2.26s, Qwen3-8B from 7.72s to 2.18s, Qwen3-14B from 12.40s to 2.50s, and Phi-4-14B from 13.81s to 2.75s. The compute savings achieved by shortening the quadratic self-attention context window far outweighed the cost of issuing two sequential forward passes per turn.

In Experiment 3, parameter-efficient fine-tuning via QLoRA (effective batch size 8 with gradient accumulation) exhibited a stable memory footprint throughout training across all backbones, completing runs in under 2 to 3 hours per adapter on a single A100 GPU without out-of-memory errors. The resulting fine-tuned modular inference executed between 2.12s (fine-tuned Llama-3.2-3B) and 3.44s (fine-tuned Phi-4-14B) per turn.

These results fully met our expectations, demonstrating that modular decomposition combined with parameter-efficient fine-tuning achieves commercial-grade latency profiles while running entirely on open-source weights.

7.2 Images or graphics showing results from the scalability testing

All tables and figures (including photographs, schemas, graphs and diagrams) should be numbered with Arabic numerals (1, 2,...n) and include a descriptive caption. Please attach the images to this form (minimum resolution 300 dpi).

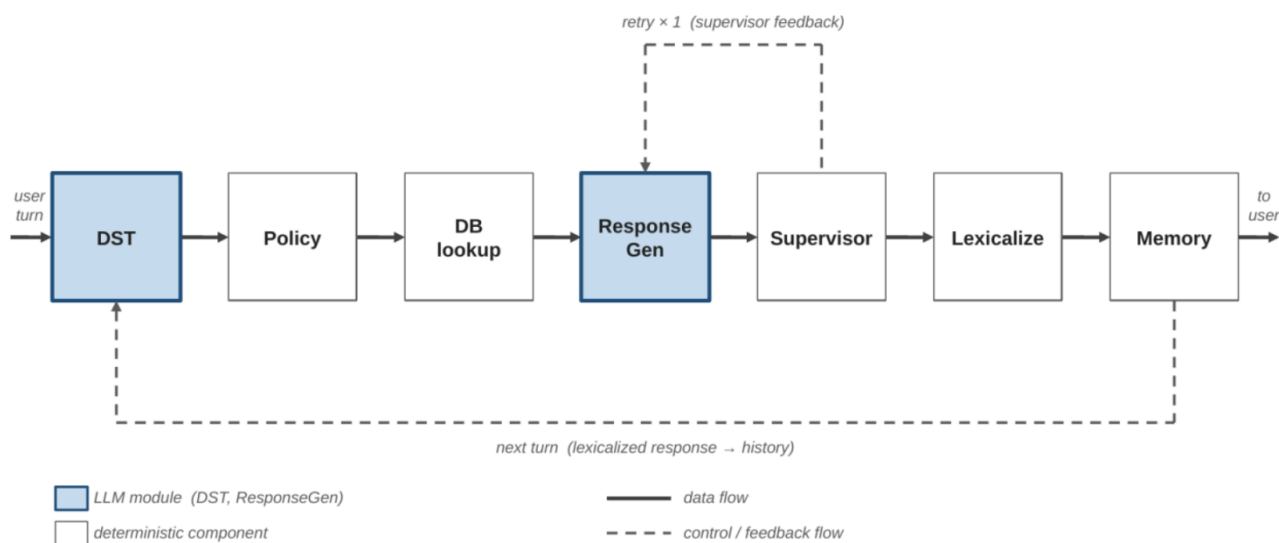


Figure 1: Modular execution pipeline (MLP4CS) deployed on Leonardo Booster. Architectural decomposition separates DST and ResponseGen into concise sub-tasks (capped at 2,048 tokens), avoiding the computational slowdown caused by long input prompts from injecting full database contexts.

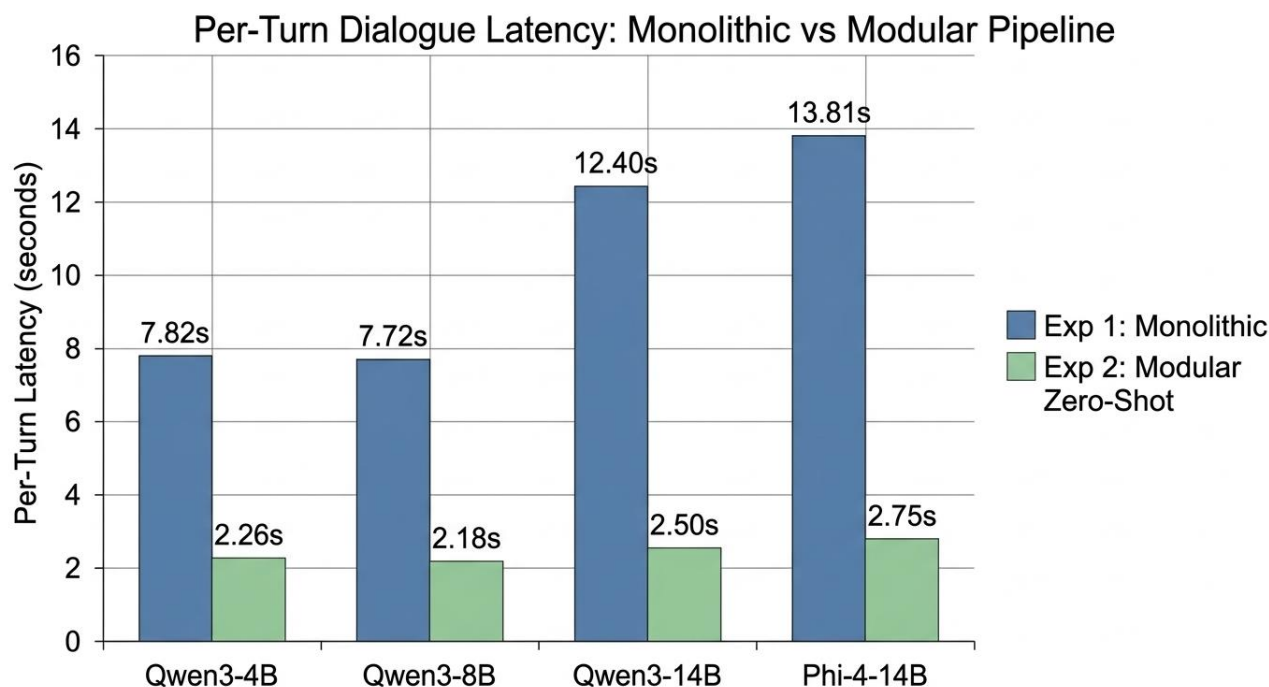


Figure 2: Per-turn dialogue latency comparison across model scales on Leonardo Booster. Architectural decomposition (Exp 2: Modular Zero-Shot) achieves a 3x to 5x latency reduction compared to the monolithic baseline (Exp 1), avoiding the high computational overhead of



processing serialized databases within long context windows.

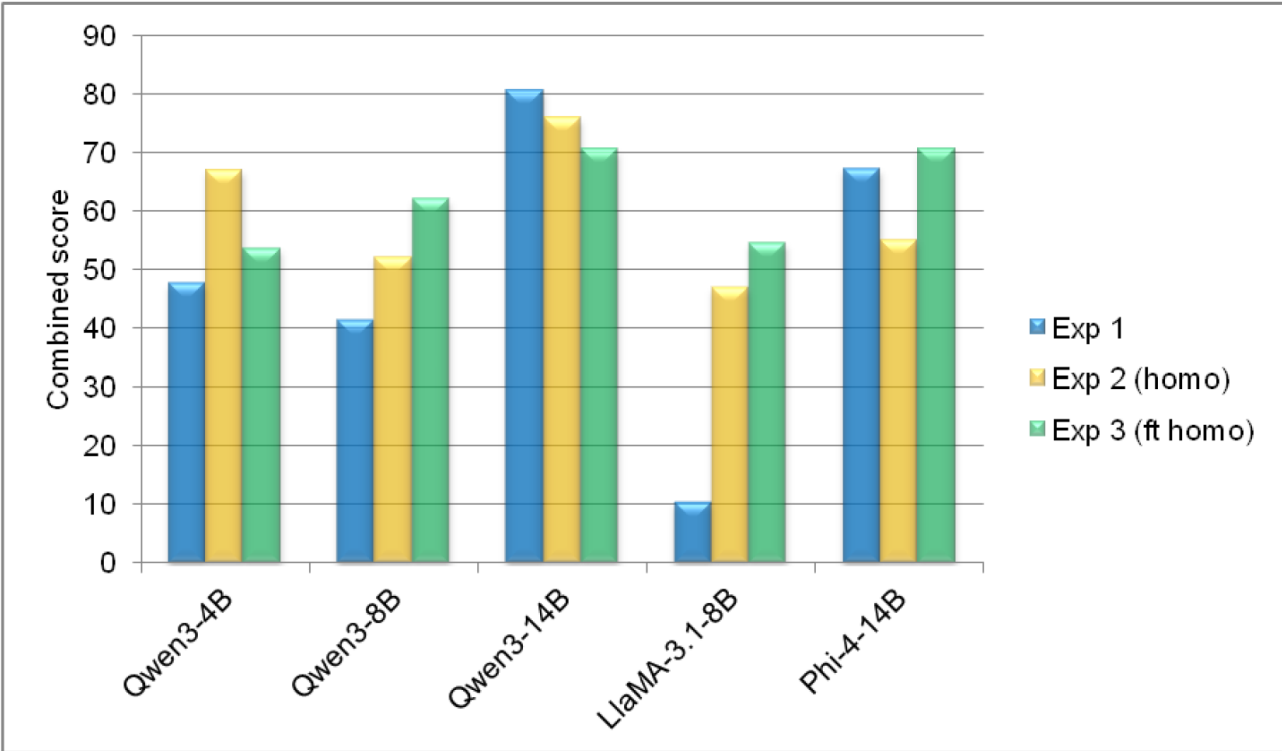


Figure 3. Performance scaling across model parameters (3B to 14B) and architectures. MultiWOZ Combined Score across Experiment 1 (Monolithic), Experiment 2 (Modular Zero-Shot), and Experiment 3 (Modular Fine-Tuned), showing how parameter scale impacts task completion under different architectural regimes.

7.3 Data to deploy scalability curves

A. Typical user test cases

Please include the data for each test case.

Number of cores	Wall clock time	Speed-up vs. the first one	Number of nodes	Number of processes
16	7.82 s/turn (Qwen3-4B Exp1)	1.00x	1	1
16	2.26 s/turn (Qwen3-4B Exp2)	3.46x	1	1



16	2.50 s/turn (Qwen3-14B Exp2)	3.13x	1	1
16	3.42 s/turn (Qwen3-14B Exp3)	2.29x	1	1

B. Strong scaling curve

Please include the data in order to deploy the scalability curve when the number of processors varies for a fixed total problem size.

Number of cores	Wall clock time	Speed-up vs the first one	Number of nodes	Number of processes
N/A	N/A	N/A	N/A	N/A

C. Weak scaling curve

Please include the data in order to deploy the scalability curve when the number of processors varies for a fixed problem size per processor.

Number of cores	Wall clock time	Speed-up vs the first one	Number of nodes	Number of processes
N/A	N/A	N/A	N/A	N/A

N/A: Single-GPU workload structure was used; multi-GPU distributed scaling was not applicable.

7.4 Publications or reports regarding the scalability testing

Please use the following format: Author(s). "Title". Publication, volume, issue, page, month year.

Georgios Xydias. "Modular LLM Pipeline for Task-Oriented Customer Service Dialogue". Master's Thesis, Department of Informatics and Telecommunications, National and Kapodistrian University of Athens, June 2026. (Repository: <https://github.com/GiX007/mlp4cs>).

8 Development and optimization

8.1 Summary of the obtained results from the enabling process



Please describe the spent effort. Which progress was achieved? Please describe in detail which enabling work was performed (porting, work on algorithms, I/O...etc.). List problems encountered, if any (maximum 500 words).

Enabling work focused on porting the customer service dialogue pipeline from a conceptual design (MAS4CS) to an optimized modular architecture (MLP4CS) on the Leonardo Booster partition, followed by role-specific parameter-efficient fine-tuning.

Primary enabling steps included:

1. **Environment Porting & I/O Setup:** Configured an isolated Python 3.10 virtual environment on Leonardo Booster with pinned, mutually compatible builds of PyTorch 2.7.0, Transformers, BitsAndBytes, and Unsloth.
2. **Architectural Optimization:** Transitioned from a monolithic single-prompt baseline (Experiment 1) to a decoupled modular pipeline (Experiments 2 and 3). In Experiment 1, full database serialization expanded prompt contexts beyond 20,000 tokens, degrading per-turn latency (12.4s to 13.8s) and causing context limit bottlenecks on models with 16K native windows (such as Phi-4-14B). Decomposing execution into dedicated Dialogue State Tracking (DST) and Response Generation (ResponseGen) modules bounded prompts to under 2,048 tokens, cutting per-turn latency by 3x to 5x (down to 2.2s to 2.8s) while eliminating memory and token limit risks.
3. **Training & Algorithmic Enablement:** Implemented single-GPU parameter-efficient fine-tuning via QLoRA (4-bit NF4 base quantization with 16-bit LoRA adapter ranks). Training routines were stabilized using 8-bit AdamW, gradient accumulation (effective batch size 8), and validation loss early stopping across the MultiWOZ 2.2 hotel and restaurant domains.
4. **Evaluation & Structured Output Standardization:** Standardized prompt serialization to guarantee train-inference alignment, and integrated the mwzeval benchmark suite alongside custom evaluators.

Problems Encountered and Resolved:

Initial monolithic runs suffered from excessive context overhead and severe latency degradation. This was resolved through architectural partitioning. Minor package dependency incompatibilities between bitsandbytes and cluster CUDA drivers were resolved by strict version pinning within the local environment.

The enabling process confirmed that fine-tuned open-source modular components achieve higher DST tracking accuracy (JGA 47.7%, Slot F1 89.5%) than commercial API baselines (Claude Haiku 4.5 JGA 45.1%, Slot F1 87.3%), while running fully locally with single-GPU execution.

8.2 Used tools for the code analysis, if applicable



Please fill in the information in the box below (e.g., Scalasca, Vampir, etc.) (maximum 500 words).

Traditional distributed profiling suites (e.g., Scalasca, Vampir) were not applicable, as jobs ran as independent single-GPU instances. Code behavior, resource usage, and execution bottlenecks were analyzed using the following tools:

- Unsloth built-in diagnostics: Monitored VRAM allocation, reserved memory overhead, and kernel speedups during 4-bit model initialization and QLoRA forward/backward passes.
- Hugging Face TRL / SFT Trainer logging: Tracked training loss, validation cross-entropy curves, and evaluation checkpoints to manage automated early stopping.
- Python standard library (time module): Built directly into the inference orchestrator to profile per-turn and per-component latency (DST inference, deterministic DB lookups, ResponseGen inference, supervisor validation, and lexicalization).
- CINECA SLURM utilities: Used post-execution to audit GPU execution time, CPU/memory efficiency, and resource utilization per batch job.

8.3 Main actions taken for optimization or improvement of codes on the EuroHPC JU systems. Which features were optimized? What were the bottlenecks? Which were the solutions (if any)?

Please fill in the information in the box below (maximum 500 words).



The primary code optimizations and algorithmic improvements on Leonardo Booster addressed context-window scaling, inference latency, GPU memory overhead, and role-specific task accuracy:

1. Prompt Partitioning and Context Bottleneck Elimination:
 - Bottleneck: The initial monolithic baseline (Experiment 1) serialized domain databases and full dialogue history into a single LLM prompt exceeding 20,000 tokens, degrading per-turn inference latency to 12.4s–13.8s on 14B models.
 - Solution: We decoupled the architecture into a modular pipeline (Experiment 2) separating Dialogue State Tracking (DST) and Response Generation (ResponseGen), bridged by deterministic database lookups and policy gates. This bounded sequence lengths to under 2,048 tokens and cut per-turn inference latency by 3x to 5x across all models (e.g., Qwen3-14B dropped from 12.40s to 2.50s per turn).
2. Memory and Precision Optimization for Fine-Tuning:
 - Bottleneck: Standard 16-bit fine-tuning of 8B to 14B parameter models typically requires over 64 GB of VRAM for optimizer states and gradients, which exceeds a single available GPU and would require multi-GPU distributed orchestration.
 - Solution: To maintain a lightweight, portable single-GPU execution model, we implemented QLoRA via Unsloth and bitsandbytes. Base models were loaded in 4-bit NormalFloat (NF4) while training 16-bit LoRA adapter weights across attention and MLP projection layers (rank 16, alpha 16). Combined with 8-bit AdamW and gradient accumulation (effective batch size 8), this capped peak VRAM well within single-GPU limits (under 25 GB even for 14B models), stabilized training, and completed runs in under 2 to 3 hours per adapter.
3. Context Window Scaling:
 - Bottleneck: Phi-4-14B has a native context limit of 16K tokens, which caused truncation errors during Experiment 1 full-database baseline evaluations.
 - Solution: We applied 2x RoPE (Rotary Position Embedding) frequency scaling within the model configuration, extending the effective context window to 32K tokens to allow complete monolithic evaluation without runtime crashes.
4. Resolving Task Asymmetry via Role Specialization:
 - Bottleneck: Modular profiling in Experiment 2 revealed dialogue state tracking as the core accuracy bottleneck across open-source models.
 - Solution: In Experiment 3, we trained dedicated task-specific LoRA adapters with strict train-inference prompt alignment. This specialization lifted tracking accuracy for local models (fine-tuned Qwen3-14B: JGA 47.7%, Slot F1 89.5%) above zero-shot commercial API baselines (Claude Haiku 4.5: JGA 45.1%, Slot F1 87.3%), achieving commercial-grade performance locally at zero API cost.

8.4 Publications or reports regarding the development and optimization

Please use the following format: Author(s). "Title". Publication, volume, issue, page, month year.



Georgios Xydias. "Modular LLM Pipeline for Task-Oriented Customer Service Dialogue". Master's Thesis, Department of Informatics and Telecommunications, National and Kapodistrian University of Athens, June 2026. (Repository: <https://github.com/GiX007/mlp4cs>).

9 Results on Input/Output

9.1 Size of the data and/or the number of files

Please fill in the information in the box below (maximum 300 words).

The project's I/O footprint consisted of caching 4-bit quantized open-source models, training datasets, and inference evaluation artifacts, stored entirely on the cluster's high-capacity \$WORK partition:

1. Model Checkpoints: Pre-quantized 4-bit weights (LLaMA-3.2-3B, LLaMA-3.1-8B, Qwen3-4B, Qwen3-8B, Qwen3-14B, and Phi-4-14B) were cached directly from Hugging Face via Unsloth. The cached weights and generated LoRA adapter checkpoints occupied approximately 50 GB of storage in total.
2. Datasets and Evaluation Outputs: The MultiWOZ 2.2 JSON files (hotel and restaurant subsets) and the resulting turn-by-turn dialogue prediction logs and metric files totaled under 500 MB across several hundred JSON and text files.

All model loading was handled via fast direct-to-GPU memory streams using Unsloth and PyTorch, remaining well within the project's disk quota.

9.2 Usage of MPI-IO features, if applicable

Please fill in the information in the box below (maximum 300 words).

N/A (MPI-IO was not used). The project relied on standard Python file operations to read and write JSON/log files, and utilized standard PyTorch/Hugging Face routines to stream model weights directly to GPU memory.

10 Main results

10.1 Conclusions about the project



Please fill in the information in the box below.

This project developed and evaluated three distinct LLM architectures for task-oriented customer service dialogue (hotel and restaurant domains of MultiWOZ 2.2) on the EuroHPC Leonardo Booster supercomputer: a monolithic baseline (Experiment 1), a zero-shot modular pipeline (Experiment 2), and a role-specific fine-tuned modular pipeline (Experiment 3). Across 29 evaluated configurations covering commercial APIs and open-source models (3B to 14B parameters), the experimental findings yield the following primary conclusions:

1. Architectural Decomposition is Conditional: Decomposing a monolithic LLM into a modular pipeline (separating Dialogue State Tracking from Response Generation) acts as a major efficiency lever. By bounding input contexts to under 2,048 tokens (down from >20,000 tokens in full-database baselines), per-turn latency dropped by 3x to 5x across open-source models (down to 2.2s–2.7s per turn), while commercial API spend fell by 80% to 90%. However, quality impact depends heavily on model capacity: decomposition substantially benefits smaller or mid-tier models that struggle with long-context instruction tracking (e.g., LLaMA-3.1-8B Combined Score increased from 10.40 to 46.98; GPT-4o-mini rose from 54.43 to 78.64), but it can slightly disadvantage high-capacity models that coordinate all tasks effectively in a single forward pass (e.g., Qwen3-14B shifted from 80.84 to 76.00).
2. Dialogue State Tracking (DST) as the Dominant Pipeline Role: Error and ablation analyses identified DST as the primary bottleneck governing end-to-end task completion. Upgrading the DST module to a more capable model yielded significantly higher system-wide gains than upgrading the Response Generator alone.
3. Specialization Closes the API Gap: Supervised parameter-efficient fine-tuning via QLoRA on Leonardo Booster (Experiment 3) successfully enabled local open-source models to surpass commercial zero-shot API baselines on structured tracking. The fine-tuned Qwen3-14B model achieved a Joint Goal Accuracy (JGA) of 47.7% and Slot F1 of 89.5% (outperforming Claude Haiku 4.5 at 45.1% JGA and 87.3% Slot F1) while executing fully on local single-GPU resources with zero API cost.
4. End-to-End Bottlenecks and Future Work: While role-specific adapter tuning resolved DST bottlenecks, overall dialogue completion remained limited by response generation quality and surface-form lexicalization errors inherited from noisy dataset annotations. Addressing this residual gap highlights dataset refinement and preference alignment (such as DPO) as the next technical milestones.

10.2 Usability of the assigned EuroHPC JU system

Please fill in the information in the box below.



The Leonardo Booster supercomputer (hosted by CINECA) provided high availability, reliable GPU hardware, and stable job scheduling throughout the project.

Key usability observations include:

1. **Hardware Performance & Stability:** The NVIDIA A100 (64 GB) GPU nodes delivered excellent computational throughput for both 4-bit quantized inference and QLoRA fine-tuning. High VRAM capacity and memory bandwidth enabled seamless execution of models up to 14B parameters without out-of-memory disruptions.
2. **Storage & File system:** The high-performance storage on the \$WORK partition provided fast read/write access, essential for caching large model checkpoints and managing multiple evaluation logs concurrently without I/O latency bottlenecks.
3. **Job Scheduling & Resource Allocation:** The SLURM workload manager was responsive and reliable. Single-GPU job requests experienced short queue wait times on the booster partition, enabling rapid, iterative testing across 29 experimental configurations.
4. **Software Environment Setup:** Installing Miniconda locally allowed us to easily create an isolated Python 3.10 environment and install all necessary AI packages without issues.

Overall, the Leonardo Booster platform proved to be a robust, high-performance environment that made the large-scale evaluation and fine-tuning experiments of this project possible.

11 Feedback and technical deployment

11.1 Feedback on the centers/EuroHPC JU access procedures

Please fill in the information in the box below (maximum 500 words).

The EuroHPC JU access submission and onboarding process through CINECA was transparent, efficient, and well-supported. Account creation, project resource allocation, and SSH credential provisioning were handled smoothly, enabling fast access to the Leonardo Booster partition.

The documentation provided for the Leonardo system (covering cluster layout, SLURM batch partitions, storage hierarchies, and environment modules) was clear and thorough. User support communications regarding scheduled maintenance windows were timely and informative. The Development Access call provided an ideal framework for this academic project, delivering the dedicated GPU compute needed to validate modular LLM pipelines and execute fine-tuning runs reliably.



11.2 Explanation of how the computer time was used compared with the work plan presented in the proposal. Justification of discrepancies, especially if the computer time was not completely used.

Please fill in the information in the box below (maximum 500 words).

The GPU compute time was used as planned to evaluate open-source models across monolithic and modular architectures (Experiments 1 and 2), followed by QLoRA fine-tuning and evaluation (Experiment 3). All planned deliverables were completed across 29 experimental configurations on MultiWOZ 2.2.

Any unused GPU hours were the direct result of two optimizations:

1. Shorter Contexts via Modular Design: Switching from monolithic prompts (>20k tokens) to modular sub-tasks (≤ 2 k tokens) cut inference latency by 3x to 5x across all models. This drastically reduced the GPU hours needed to evaluate the test sets.
2. Fast Single-GPU QLoRA Fine-Tuning: Using 4-bit NF4 quantization, 8-bit AdamW, and early stopping allowed each adapter to converge in under 2 to 3 hours on a single A100. This avoided the need for multi-GPU training and reduced overall compute consumption.

All proposed research goals were met, with efficiency improvements allowing us to complete the experiments using fewer GPU hours than originally budgeted.

11.3 Willingness to apply to EuroHPC JU Access Modes in the future

Please fill in the information in the box below (maximum 500 words).

Yes, absolutely. Having access to the Leonardo Booster supercomputer through EuroHPC JU was essential for completing my Master's thesis. The high-performance NVIDIA A100 GPUs gave me the compute power needed to run experiments and fine-tune open-source models that I could never have run on regular local machines.

The application and the onboarding process was smooth, and the system was reliable throughout the work. If the opportunity arises in the future, I would gladly apply again whether for continued research in natural language processing or to prototype and scale production-ready AI solutions.